

Bellman Expectation Equation

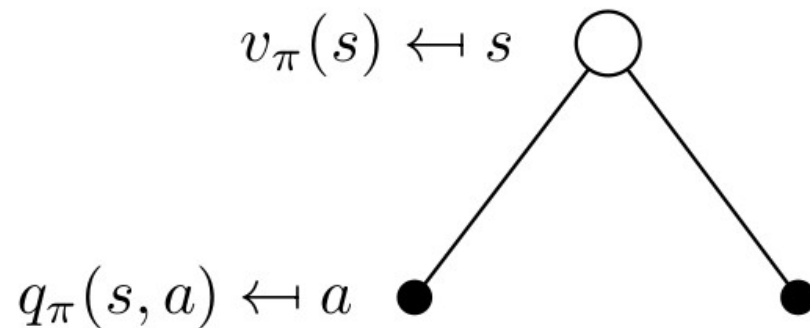
- The state-value function can again be decomposed into immediate reward plus discounted value of successor state,

$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s]$$

- The action-value function can similarly be decomposed,

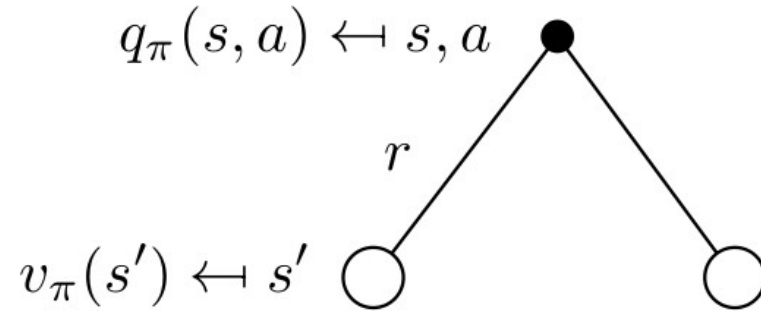
$$q_{\pi}(s, a) = \mathbb{E}_{\pi} [R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

Bellman Expectation Equation for v_π



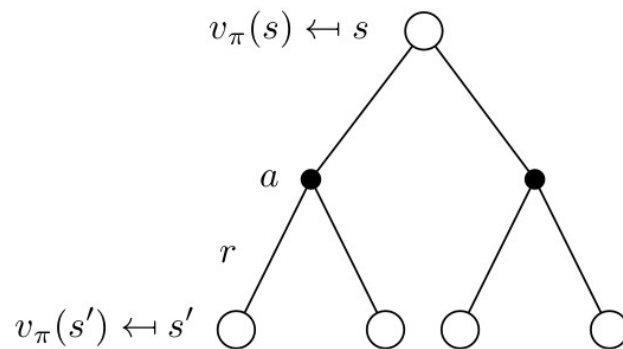
$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s, a)$$

Bellman Expectation Equation for q_π



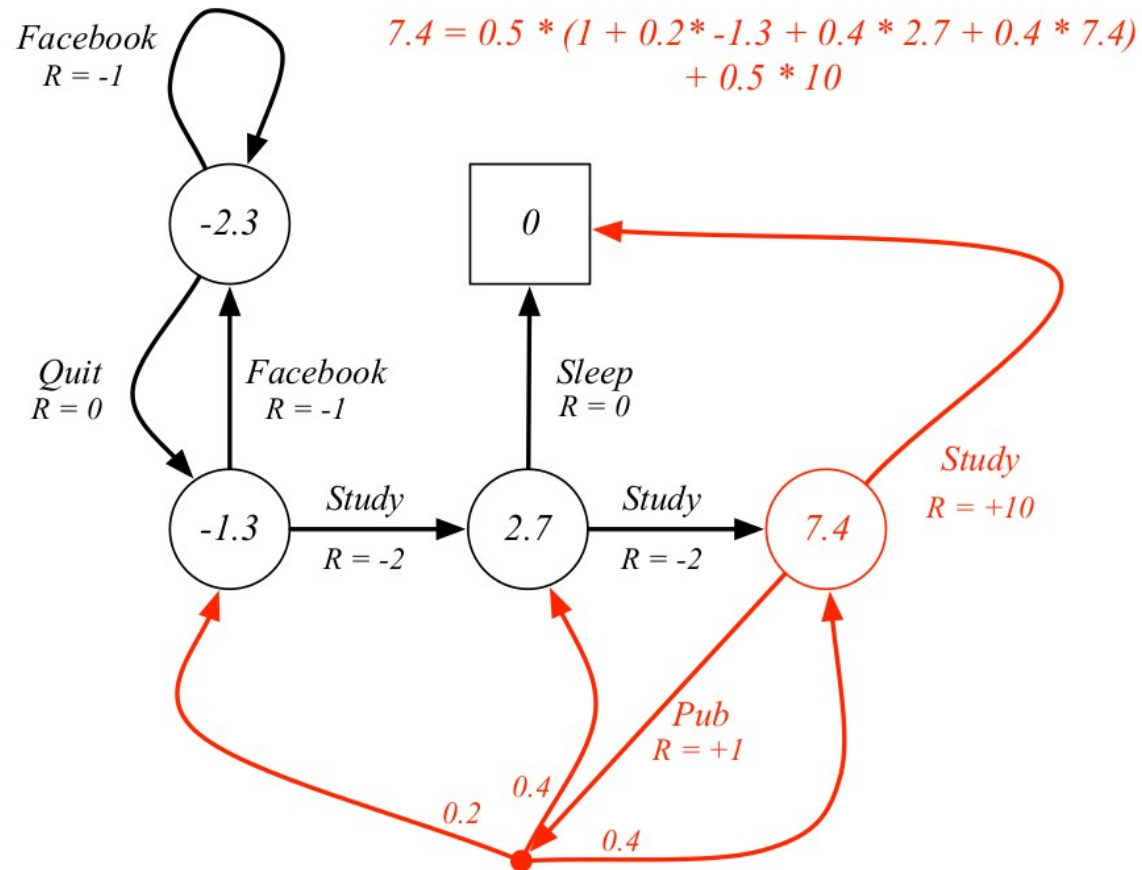
$$q_\pi(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s')$$

Bellman Expectation Equation for v_π



$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \right)$$

Bellman Expectation Equation in Student MDP



Bellman Expectation Equation (Matrix Form)

- The Bellman expectation equation can be expressed concisely using the induced MRP,

$$v_{\pi} = \mathcal{R}^{\pi} + \gamma \mathcal{P}^{\pi} v_{\pi}$$

With direct solution

$$v_{\pi} = (I - \gamma \mathcal{P}^{\pi})^{-1} \mathcal{R}^{\pi}$$

Optimal Value Function

The **optimal** *state-value function* $v_*(s)$ is the maximum value function over all policies

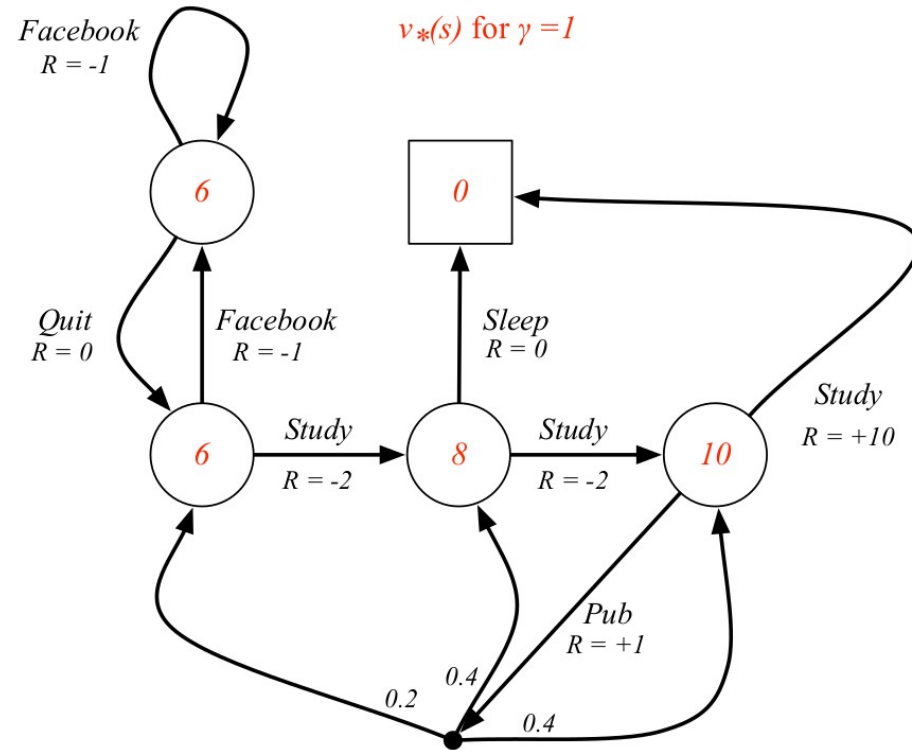
$$v_*(s) = \max_{\pi} v_{\pi}(s)$$

The **optimal** *action-value function* $q_*(s, a)$ is the maximum action-value function over all policies

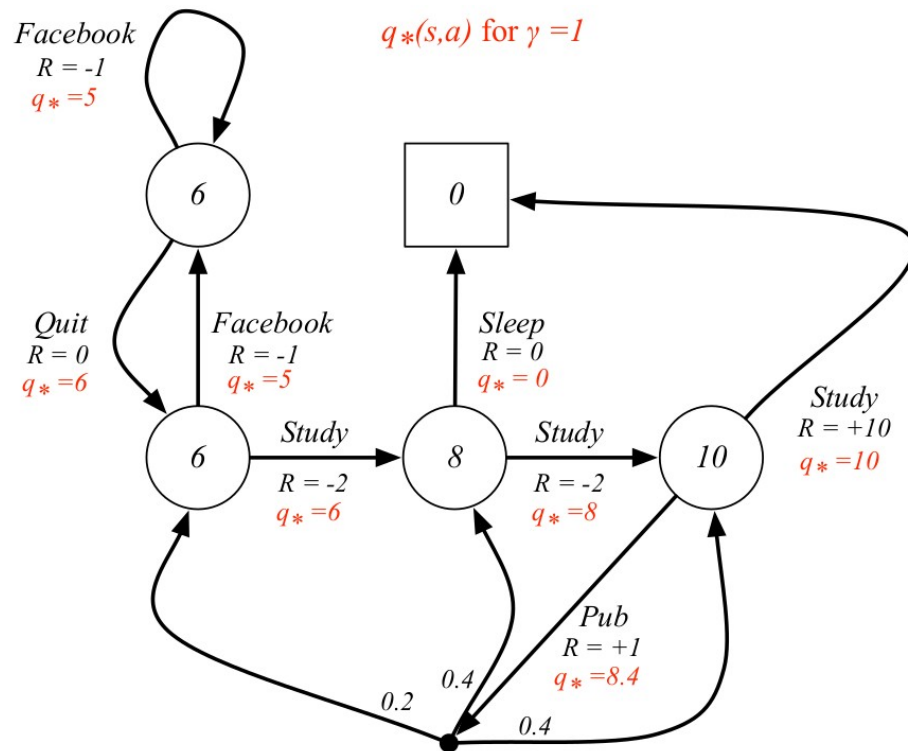
$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

The optimal value function specifies the best possible performance in the MDP. An MDP is “solved” when we know the optimal value function.

Optimal Value Function for Student MDP



Optimal Action-Value Function for Student MDP



Optimal Policy

Define a partial ordering over policies

$$\pi \geq \pi' \text{ if } v_{\pi}(s) \geq v_{\pi'}(s), \forall s$$

Theorem:

For any Markov Decision Process

There exists an optimal policy π^* that is better than or equal to all other policies,
 $\pi^* \geq \pi, \forall \pi$

All optimal policies achieve the optimal value function, $v_{\pi^*}(s) = v^*(s)$

All optimal policies achieve the optimal action-value function, $q_{\pi^*}(s, a) = q^*(s, a)$

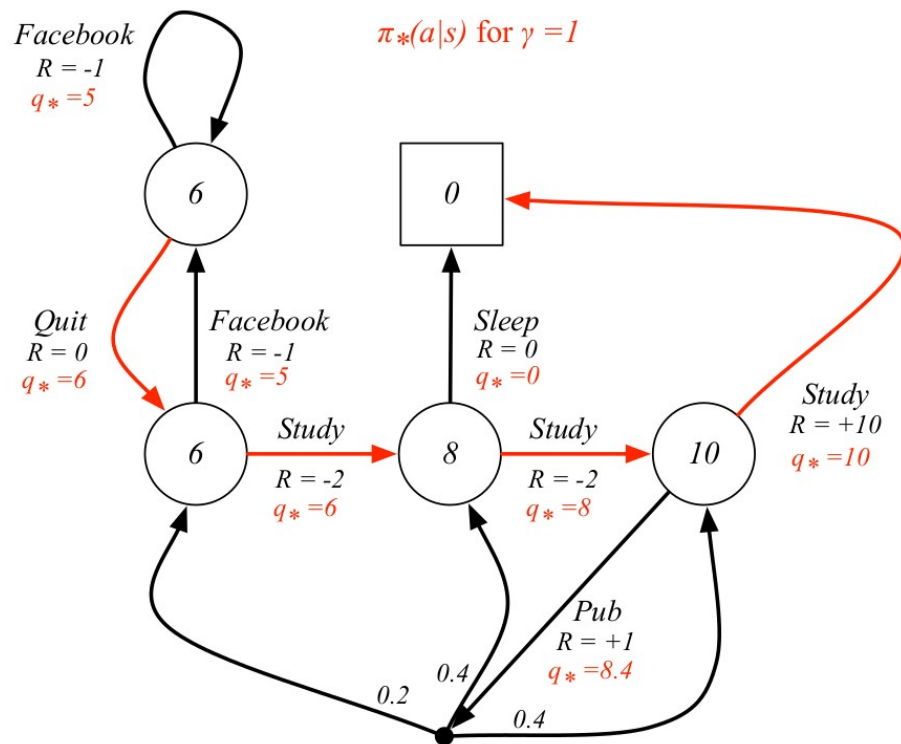
Finding an Optimal Policy

An optimal policy can be found by maximising over $q_*(s, a)$,

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \operatorname{argmax}_{a \in \mathcal{A}} q_*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

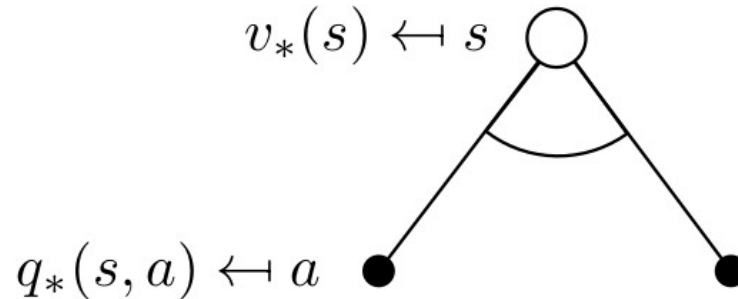
There is always a deterministic optimal policy for any MDP
If we know $q_*(s, a)$, we immediately have the optimal policy

Optimal Policy for Student MDP



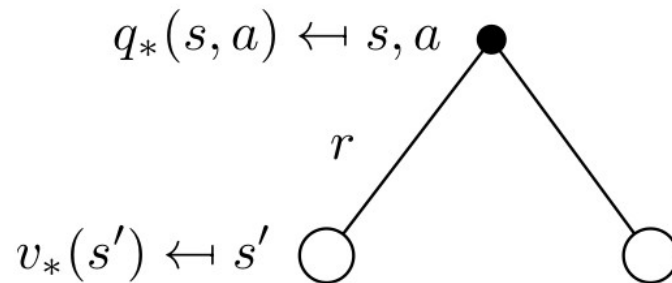
Bellman Optimality Equation for v_*

The optimal value functions are recursively related by the Bellman optimality equations:



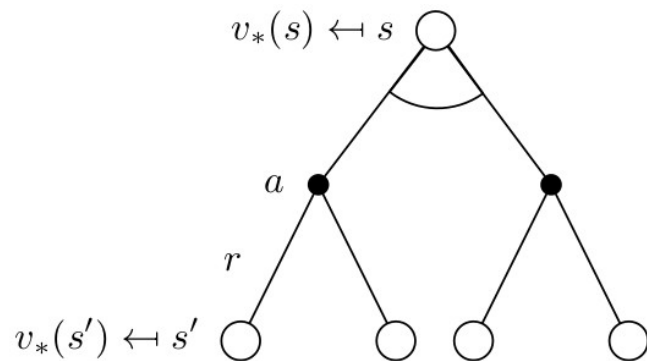
$$v_*(s) = \max_a q_*(s, a)$$

Bellman Optimality Equation for q_*



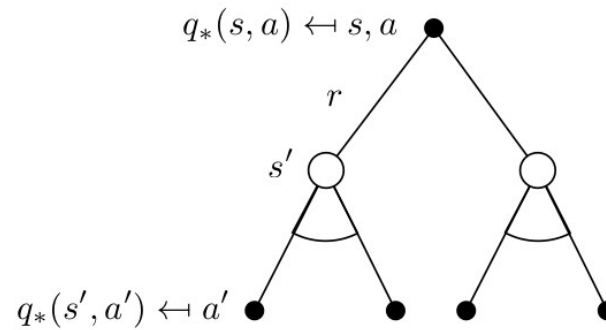
$$q_*(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

Bellman Optimality Equation for v_*



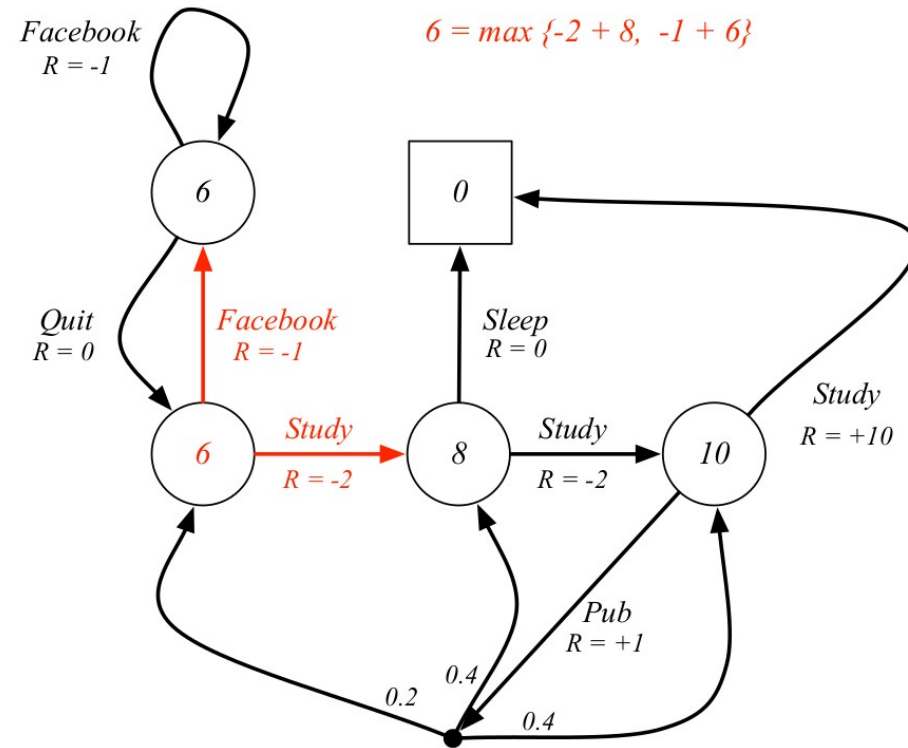
$$v_*(s) = \max_a \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

Bellman Optimality Equation for q_*



$$q_*(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} q_*(s', a')$$

Bellman Optimality Equation in Student MDP



Solving the Bellman Optimality Equation

- Bellman Optimality Equation is non-linear
- No closed form solution (in general)
- Many iterative solution methods
 - Value Iteration
 - Policy Iteration
 - Q-learning
 - Sarsa

Planning by Dynamic Programming

Longest common subsequence problem

Dynamic Programming

Example,

$S = ABAZDC$

$T = BACBAD$

Initialization

		B	A	C	B	A	D
		0	0	0	0	0	0
A	0						
B	0						
A	0						
Z	0						
D	0						
C	0						

Longest common subsequence problem

Dynamic Programming

Example,

$S = \text{ABAZDC}$

$T = \text{BACBAD}$

$$M_{i,j} = \text{MAXIMUM} [M_{i-1,j-1} + S, M_{i-1,j}, M_{i,j-1}]$$

		B	A	C	B	A	D
		0	0	0	0	0	0
A	0	0	1	1	1	1	1
B	0	1	1	1	2	2	2
A	0	1	2	2	2	3	3
Z	0	1	2	2	2	3	3
D	0	1	2	2	2	3	4
C	0	1	2	3	3	3	4

Initialization
Scoring

Longest common subsequence problem

Dynamic Programming

Example,

$S = \text{ABAZDC}$

$T = \text{BACBAD}$

$$M_{i,j} = \text{MAXIMUM} [M_{i-1,j-1} + S, M_{i-1,j}, M_{i,j-1}]$$

Solution???

		B	A	C	B	A	D
		0	0	0	0	0	0
A	0	0	1	1	1	1	1
B	0	1	1	1	2	2	2
A	0	1	2	2	2	3	3
Z	0	1	2	2	2	3	3
D	0	1	2	2	2	3	4
C	0	1	2	3	3	3	4

Initialization
Scoring
Alignment

Multiple
Possibilities

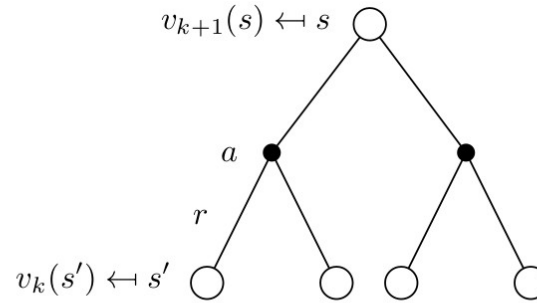
Planning by Dynamic Programming

- Dynamic programming assumes full knowledge of the MDP
- It is used for planning in an MDP
- For prediction:
 - Input: MDP $\langle S, A, P, R, \gamma \rangle$ and policy π
 - or: MRP $\langle S, P^\pi, R^\pi, \gamma \rangle$
 - Output: value function v_π
- Or for control:
 - Input: MDP $\langle S, A, P, R, \gamma \rangle$
 - Output: optimal value function v_*
 - and: optimal policy π_*

Iterative Policy Evaluation

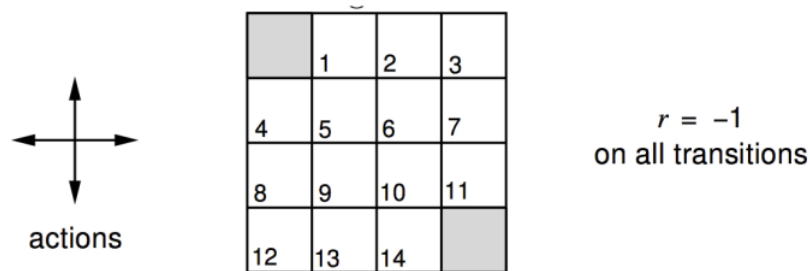
- Problem: evaluate a given policy π
- Solution: iterative application of Bellman expectation backup
- $V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V_\pi$
- Using synchronous backups,
 - At each iteration $k+1$
 - For all states $s \in S$
 - Update $v_{k+1}(s)$ from $v_k(s')$
 - where s' is a successor state of s

Iterative Policy Evaluation



$$v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$
$$\mathbf{v}^{k+1} = \mathcal{R}^\pi + \gamma \mathcal{P}^\pi \mathbf{v}^k$$

Evaluating a Random Policy in the Small Gridworld



Undiscounted episodic MDP ($\gamma = 1$)

Nonterminal states 1, ..., 14

One terminal state (shown twice as shaded squares)

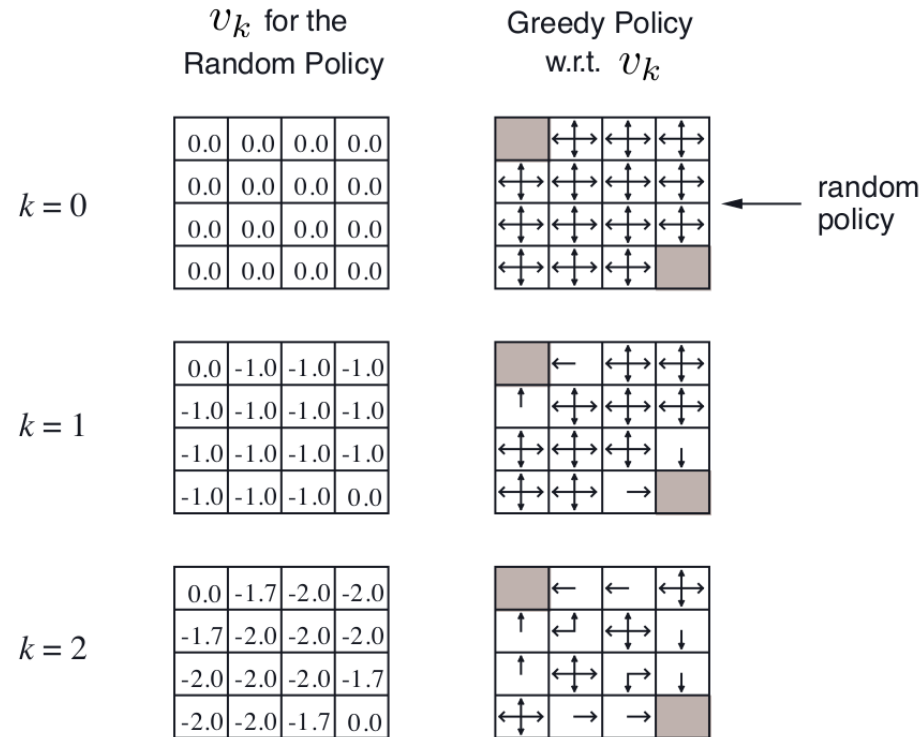
Actions leading out of the grid leave state unchanged

Reward is -1 until the terminal state is reached

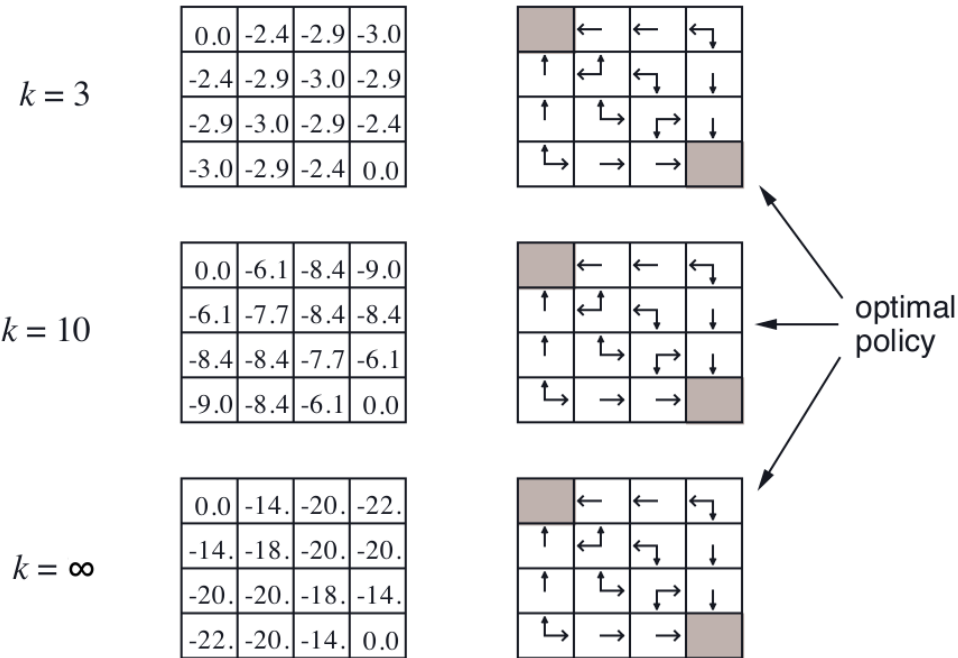
Agent follows uniform random policy

$$\pi(n|\cdot) = \pi(e|\cdot) = \pi(s|\cdot) = \pi(w|\cdot) = 0.25$$

Iterative Policy Evaluation in Small Gridworld



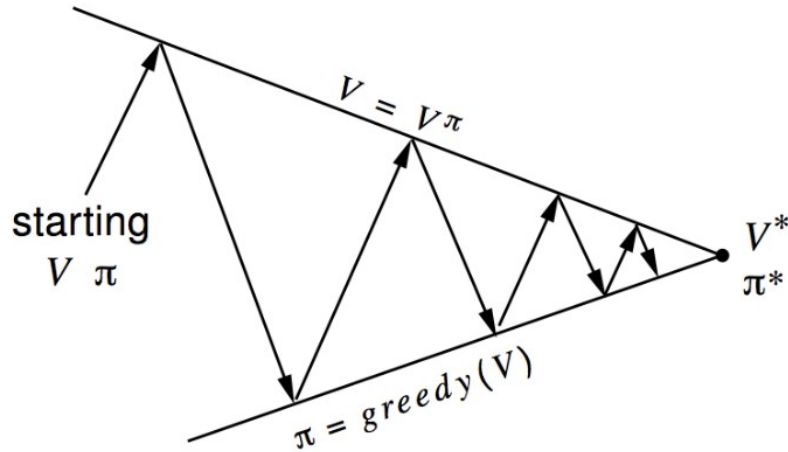
Iterative Policy Evaluation in Small Gridworld



How to Improve a Policy

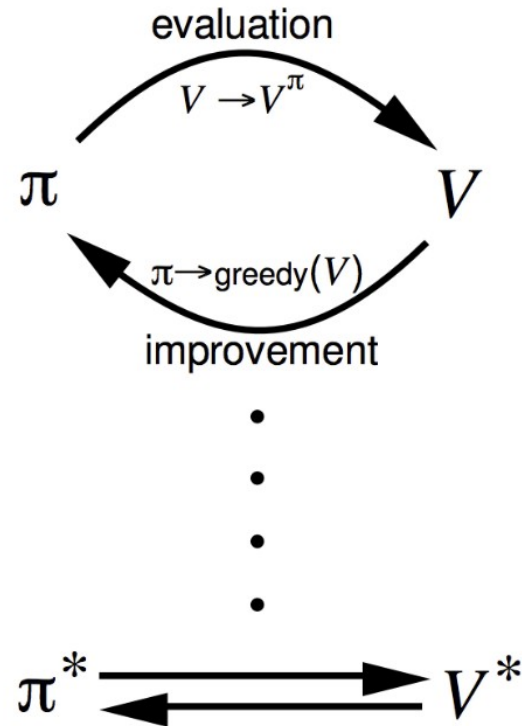
- Given a policy π
 - Evaluate the policy π
$$v_{\pi}(s) = E [R_{t+1} + \gamma R_{t+2} + \dots | S_t = s]$$
 - Improve the policy by acting greedily with respect to v_{π}
$$\pi' = \text{greedy}(v_{\pi})$$
- In Small Gridworld improved policy was optimal, $\pi' = \pi^*$
- In general, need more iterations of improvement / evaluation
- But this process of policy iteration always converges to π^*

Policy Iteration



Policy evaluation Estimate v_π
 Iterative policy evaluation

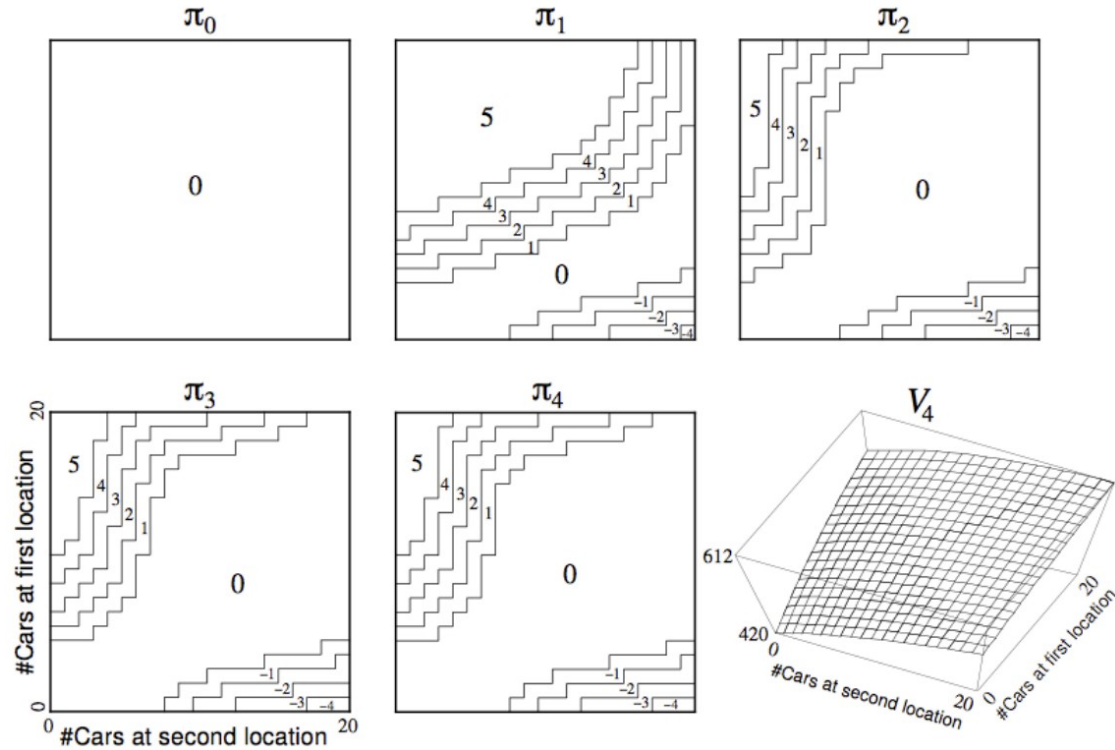
Policy improvement Generate $\pi' \geq \pi$
 Greedy policy improvement



Jack's Car Rental

- States: Two locations, maximum of 20 cars at each
- Actions: Move up to 5 cars between locations overnight
- Reward: \$10 for each car rented (must be available)
- Transitions: Cars returned and requested randomly
 - Poisson distribution, n returns/requests with prob $(\lambda^n/n!)e^{-\lambda}$
 - 1st location: average requests = 3, average returns = 3
 - 2nd location: average requests = 4, average returns = 2

Policy Iteration in Jack's Car Rental



Policy Improvement

- Consider a deterministic policy, $a = \pi(s)$
- We can improve the policy by acting greedily

$$\pi'(s) = \operatorname{argmax}_{a \in A} q_{\pi}(s, a)$$

- This improves the value from any state s over one step,
 $q_{\pi}(s, \pi'(s)) = \max_{a \in A} q_{\pi}(s, a) \geq q_{\pi}(s, \pi(s)) = v_{\pi}(s)$
- It therefore improves the value function, $v_{\pi'}(s) \geq v_{\pi}(s)$

$$\begin{aligned} v_{\pi}(s) &\leq q_{\pi}(s, \pi'(s)) = E_{\pi'}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s] \\ &\leq E_{\pi'}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] \\ &\leq E_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 q_{\pi}(S_{t+2}, \pi'(S_{t+2})) \mid S_t = s] \\ &\leq E_{\pi'}[R_{t+1} + \gamma R_{t+2} + \dots \mid S_t = s] = v_{\pi'}(s) \end{aligned}$$

Policy Improvement

- If improvements stop,

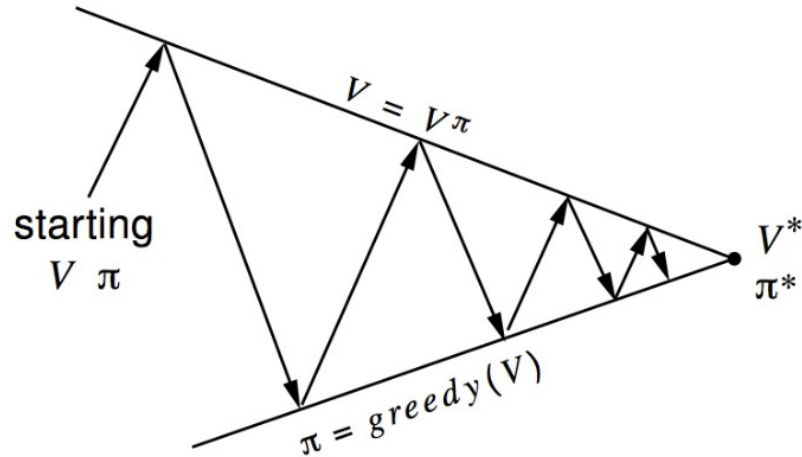
$$q_{\pi}(s, \pi'(s)) = \max_{a \in A} q_{\pi}(s, a) = q_{\pi}(s, \pi(s)) = v_{\pi}(s)$$

- Then the Bellman optimality equation has been satisfied
 - $v_{\pi}(s) = \max_{a \in A} q_{\pi}(s, a)$
- Therefore $v_{\pi}(s) = v_*(s)$ for all $s \in S$
- so π is an optimal policy

Modified Policy Iteration

- Does policy evaluation need to converge to v_π ?
- Or should we introduce a stopping condition
 - e.g. ϵ -convergence of value function
- Or simply stop after k iterations of iterative policy evaluation?
- For example, in the small gridworld $k = 3$ was sufficient to achieve optimal policy
- Why not update policy every iteration? i.e. stop after $k = 1$
 - This is equivalent to value iteration (next section)

Generalised Policy Iteration

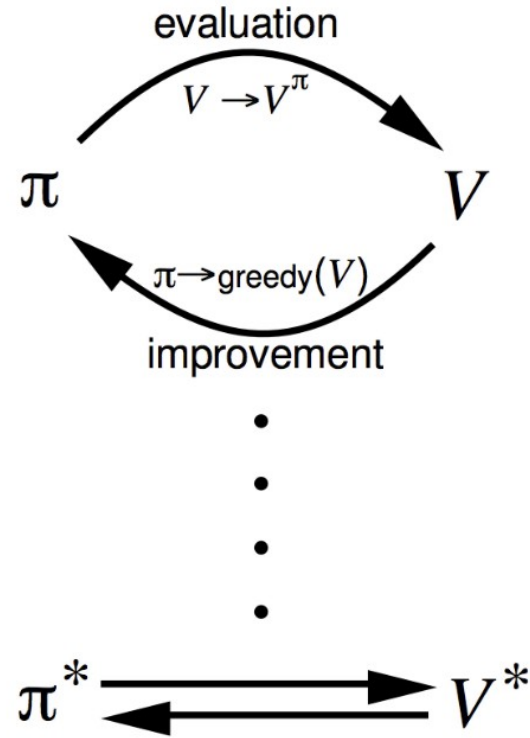


Policy evaluation Estimate v_π

Any policy evaluation algorithm

Policy improvement Generate $\pi' \geq \pi$

Any policy improvement algorithm



Principle of Optimality

- Any optimal policy can be subdivided into two components:
 - An optimal first action A_*
 - Followed by an optimal policy from successor state S'
- Theorem (Principle of Optimality)
 - A policy $\pi(a|s)$ achieves the optimal value from state s , $v_\pi(s) = v_*(s)$, if and only if
 - For any state s' reachable from s
 - π achieves the optimal value from state s' , $v_\pi(s') = v_*(s')$

Deterministic Value Iteration

If we know the solution to subproblems $v_*(s')$

Then solution $v_*(s)$ can be found by one-step lookahead

$$v_*(s) \leftarrow \max_{a \in \mathcal{A}} \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

The idea of value iteration is to apply these updates iteratively

Intuition: start with final rewards and work backwards

Still works with loopy, stochastic MDPs

Example: Shortest Path

g			

Problem

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

V_1

0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

V_2

0	-1	-2	-2
-1	-2	-2	-2
-2	-2	-2	-2
-2	-2	-2	-2

V_3

0	-1	-2	-3
-1	-2	-3	-3
-2	-3	-3	-3
-3	-3	-3	-3

V_4

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-4
-3	-4	-4	-4

V_5

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-5

V_6

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-6

V_7

Value Iteration

Problem: find optimal policy π

Solution: iterative application of Bellman optimality backup

$v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_*$

Using synchronous backups

- At each iteration $k + 1$

- For all states $s \in \mathcal{S}$

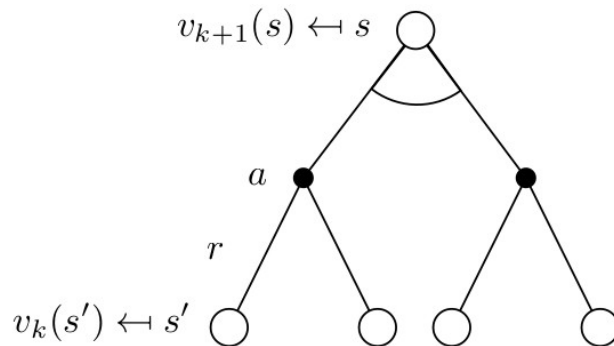
- Update $v_{k+1}(s)$ from $v_k(s')$

Convergence to v_* will be proven later

Unlike policy iteration, there is no explicit policy

Intermediate value functions may not correspond to any policy

Value Iteration



$$v_{k+1}(s) = \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$

$$\mathbf{v}_{k+1} = \max_{a \in \mathcal{A}} \mathcal{R}^a + \gamma \mathcal{P}^a \mathbf{v}_k$$

Synchronous Dynamic Programming Algorithms

Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Equation + Greedy Policy Improvement	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

Algorithms are based on state-value function $v_{\pi}(s)$ or $v^*(s)$
Complexity $O(mn^2)$ per iteration, for m actions and n states
Could also apply to action-value function $q_{\pi}(s, a)$ or $q^*(s, a)$
Complexity $O(m^2n^2)$ per iteration

Asynchronous Dynamic Programming

- DP methods described so far used synchronous backups
- i.e. all states are backed up in parallel
- Asynchronous DP backs up states individually, in any order
- For each selected state, apply the appropriate backup
- Can significantly reduce computation
- Guaranteed to converge if all states continue to be selected

In-Place Dynamic Programming

Synchronous value iteration stores two copies of value function
for all s in $\mathcal{S}$

$$v_{new}(s) \leftarrow \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_{old}(s') \right)$$

$$v_{old} \leftarrow v_{new}$$

In-place value iteration only stores one copy of value function
for all s in $\mathcal{S}$

$$v(s) \leftarrow \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v(s') \right)$$

Prioritised Sweeping

Use magnitude of Bellman error to guide state selection, e.g.

$$\left| \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v(s') \right) - v(s) \right|$$

Backup the state with the largest remaining Bellman error

Update Bellman error of affected states after each backup

Requires knowledge of reverse dynamics (predecessor states)

Can be implemented efficiently by maintaining a priority queue

Real-Time Dynamic Programming

Idea: only states that are relevant to agent

Use agent's experience to guide the selection of states

After each time-step S_t, A_t, R_{t+1}

Backup the state S_t

$$v(S_t) \leftarrow \max_{a \in \mathcal{A}} \left(\mathcal{R}_{S_t}^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{S_t s'}^a v(s') \right)$$

Full-Width Backups

DP uses *full-width* backups

For each backup (sync or async)

- Every successor state and action is considered
- Using knowledge of the MDP transitions and reward function

DP is effective for medium-sized problems (millions of states)

For large problems DP suffers Bellman's *curse of dimensionality*

- Number of states $n = |\mathcal{S}|$ grows exponentially with number of state variables

Even one backup can be too expensive

